Day 64 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 10, Day 2 reference

Watch: CS231n Winter 2016: Lecture 10: Recurrent Neural Networks, Image Captioning, LSTM (Andrej Karpathy) | Implementing LSTM in numpy from scratch! (Ather Cheema)

`day64_lstm_forward_pass.py`, segment by segment, in the order `main()` uses things. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula; the green **SYSTEM DIAGRAM** and the graphs are the ones the script saved. A glossary is at the end.

MATH BOX LEGEND

Symbols: x_t = input at step t · h, c = hidden and cell state · m_t = mask (1 real, 0 padding) · W_x, W_h = the input rows and the hidden rows of W · D, H = input and hidden size

Segment 0 — Imports

Imports

CODE

```
import time
import numpy as np
from threadpoolctl import threadpool_info, threadpool_limits
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
np.random.seed(42)
```

- `from threadpoolctl import threadpool_info, threadpool_limits` — a small library (installed with scikit-learn) that reports and limits how many CPU threads NumPy's matrix library uses.
- `from matplotlib.patches import FancyBboxPatch` — rounded boxes for the unrolled diagram.
- `np.random.seed(42)` — fixes the global random numbers; each experiment also has its own seeded generator.

Segment 1 — From Day 63

sigmoid, lstm_step_forward, init_lstm (verbatim from Day 63)

MATH

$$c = f \cdot c_{old} + i \cdot g, \quad h = o \cdot \tanh(c)$$

CODE

```
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))
def lstm_step_forward(x_t, h_prev, c_prev, W, b):
    H = h_prev.shape[1]
    a = np.concatenate([x_t, h_prev], axis=1) @ W + b
    f = sigmoid(a[:, 0 * H:1 * H])
    i = sigmoid(a[:, 1 * H:2 * H])
    g = np.tanh(a[:, 2 * H:3 * H])
    o = sigmoid(a[:, 3 * H:4 * H])
    c = f * c_prev + i * g
    h = o * np.tanh(c)
    return h, c, (f, i, g, o)
def init_lstm(input_dim, hidden_dim, seed, forget_bias=0.0, weight_scale=1.0):
    rng = np.random.RandomState(seed)
    W = rng.randn(input_dim + hidden_dim, 4 * hidden_dim) * weight_scale / np.sqrt(input_dim +
        hidden_dim)
    b = np.zeros(4 * hidden_dim)
    b[:hidden_dim] = forget_bias
    return W, b
```

- `verbatim from Day 63` — copied unchanged and labelled as such, so today's checks build on a function already tested.
- `np.concatenate([x_t, h_prev], axis=1) @ W + b` — all four gates' raw scores in one product.
- `a[:, 0 * H:1 * H]` — the forget gate's block of columns; then input, candidate, output.
- `return h, c, (f, i, g, o)` — the gates come back too: today's loop stores them for the backward pass.

Segment 2 — The forward pass

lstm_forward

MATH

$(h_t, c_t) = step(x_t, h_{t-1}, c_{t-1})$ *on real steps;* $h_t = h_{t-1}, c_t = c_{t-1}$ *on padding*

CODE

```
def lstm_forward(X, h0, c0, W, b, mask=None):
    m, T, D = X.shape
    H = h0.shape[1]
    if mask is None:
        mask = np.ones((m, T))
    Hs, Cs = np.zeros((m, T, H)), np.zeros((m, T, H))
    F, I, G, O = (np.zeros((m, T, H)) for _ in range(4))
    h, c = h0, c0
    for t in range(T):
        h_new, c_new, (f, i, g, o) = lstm_step_forward(X[:, t, :], h, c, W, b)
        m_t = mask[:, t:t + 1]
        h = m_t * h_new + (1.0 - m_t) * h
        c = m_t * c_new + (1.0 - m_t) * c
        Hs[:, t], Cs[:, t] = h, c
        F[:, t], I[:, t], G[:, t], O[:, t] = f, i, g, o
    cache = dict(X=X, h0=h0, c0=c0, W=W, b=b, mask=mask, H=Hs, C=Cs, F=F, I=I, G=G, O=O)
    return Hs, Cs, cache
```

- `mask=None` — an optional argument; None means “not given”.
- `if mask is None: mask = np.ones((m, T))` — no mask given: treat every step as real.
- `F, I, G, O = (np.zeros((m, T, H)) for _ in range(4))` — a generator expression makes four **separate** arrays, unpacked into four names.
- `h_new, c_new, (f, i, g, o) = lstm_step_forward(...)` — nested unpacking of the step's three return values.
- `mask[:, t:t + 1]` — column t kept 2-D, (m, 1), so it multiplies every hidden unit of each row.
- `m_t * h_new + (1.0 - m_t) * h` — real step: the new state; padding: the old one. Done for h and again for c.
- `Hs[:, t], Cs[:, t] = h, c` — store two arrays in one line.
- `dict(X=X, h0=h0, ...)` — the cache: a dictionary whose keys are the names given as keyword arguments.

lstm_forward_fast

MATH

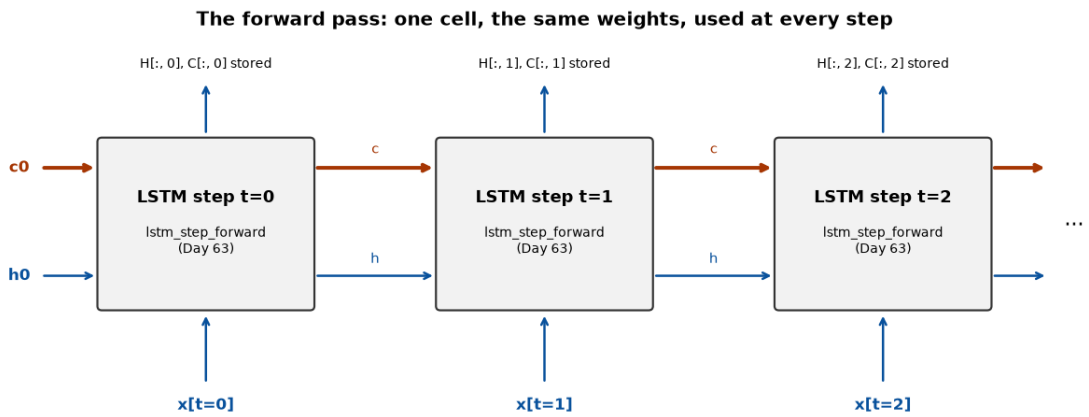
$$A = X W_x + b \text{ for all steps at once;} \quad a_t = A_t + h_{t-1} W_h$$

CODE

```
def lstm_forward_fast(X, h0, c0, W, b):
    m, T, D = X.shape
    H = h0.shape[1]
    Wx, Wh = W[:D], W[D:]
    A = (X.reshape(m * T, D) @ Wx + b).reshape(m, T, 4 * H)
    A = A.transpose(1, 0, 2).copy()
    Hs, Cs = np.zeros((m, T, H)), np.zeros((m, T, H))
    h, c = h0, c0
    for t in range(T):
        a = A[t] + h @ Wh
        f, i = sigmoid(a[:, :H]), sigmoid(a[:, H:2 * H])
        g, o = np.tanh(a[:, 2 * H:3 * H]), sigmoid(a[:, 3 * H:])
        c = f * c + i * g
        h = o * np.tanh(c)
        Hs[:, t], Cs[:, t] = h, c
    return Hs, Cs
```

- `W[:D], W[D:]` — the rows that multiply x, and the rows that multiply h.
- `X.reshape(m * T, D) @ Wx + b` — every step of every sequence as one row: one product for all of them.
- `.reshape(m, T, 4 * H)` — fold the rows back into (examples, steps, 4H).
- `A.transpose(1, 0, 2).copy()` — swap the first two axes so time comes first; .copy() lays the result out contiguously.
- `A[t] + h @ Wh` — the precomputed input part plus the only product that depends on the previous step.
- `sigmoid(a[:, :H]), sigmoid(a[:, H:2 * H])` — two gates computed and unpacked in one line.

SYSTEM DIAGRAM



lstm_forward as a picture: one step function, called once per position, handing c and h to the next call.

Segment 3 — demo_forward_checks

Checking against hand-chained steps

CODE

```
Hs, Cs, cache = lstm_forward(X, h0, c0, W, b)
h1, c1, _ = lstm_step_forward(X[:, 0], h0, c0, W, b)
h2, c2, _ = lstm_step_forward(X[:, 1], h1, c1, W, b)
h3, c3, _ = lstm_step_forward(X[:, 2], h2, c2, W, b)
print(f"\n{'step':>5}{'max |h loop - h by hand|':>27}{'max |c loop - c by hand|':>27}")
for t, (h_ref, c_ref) in enumerate([(h1, c1), (h2, c2), (h3, c3)]):
    print(f"{t:>5}{np.max(np.abs(Hs[:, t] - h_ref)):>27.2e}{np.max(np.abs(Cs[:, t] -
        c_ref)):>27.2e}")
```

- `h2, c2, _ = lstm_step_forward(X[:, 1], h1, c1, W, b)` — step 1 by hand, fed the states step 0 returned; _ discards the gates.
- `X[:, 1]` — with a 3-D array, one integer index removes that axis: (m, T, D) becomes (m, D).
- `enumerate([(h1, c1), (h2, c2), (h3, c3)])` — position and pair together; the pair is unpacked into h_ref, c_ref.
- `np.max(np.abs(Hs[:, t] - h_ref))` — the largest disagreement anywhere in step t.

Segment 4 — demo_masking

Three ways to handle padding

CODE

```
batch = np.zeros((2, 8, D)); batch[0, :3] = short[0]
batch[1] = rng.randn(8, D)
mask = np.array([[1.0] * 3 + [0.0] * 5, [1.0] * 8])
Hm, Cm, _ = lstm_forward(batch, zeros(2), zeros(2), W, b, mask=mask)
Hn, Cn, _ = lstm_forward(batch, zeros(2), zeros(2), W, b)
h, c = zeros(2), zeros(2)
for t in range(8):
    h_new, c, _ = lstm_step_forward(batch[:, t], h, c, W, b)
    h = mask[:, t:t + 1] * h_new + (1 - mask[:, t:t + 1]) * h
```

- `zeros = lambda m: np.zeros((m, H))` — (just above) a one-line function returning zero states for m examples.
- `batch[0, :3] = short[0]` — write the short sequence into the first 3 steps of row 0; the rest stays zero.
- `np.array([[1.0] * 3 + [0.0] * 5, [1.0] * 8])` — list repetition and joining: [1,1,1,0,0,0,0,0] and eight 1s.
- `h_new, c, _ = lstm_step_forward(...)` — the deliberate mistake: c is replaced at every step, padding or not.

Segment 5 — demo_fast_forward

best_time and the thread-controlled timing

CODE

```
def best_time(fn, *args, repeats=7):
    times = []
    for _ in range(repeats):
        start = time.perf_counter()
        fn(*args)
        times.append(time.perf_counter() - start)
    return min(times)
blas = [x for x in threadpool_info() if x["user_api"] == "blas"]
print(f"\nmatrix library: {blas[0]['internal_api']}, using {blas[0]['num_threads']} threads by
    default")
print(f"\n{'':>21}{'---- default threads ----':>27}{'------ one thread ------':>27}")
print(f"{'D':>4}{'H':>5}{'batch':>6}{'steps':>6}{'loop':>9}{'precomp':>9}{'speed-up':>9}"
      f"{'loop':>9}{'precomp':>9}{'speed-up':>9}")
rows = []
for D, H, m, T in [(1, 16, 200, 200), (32, 32, 32, 100), (64, 64, 32, 100), (128, 128, 32, 100),
    (300, 128, 64, 50)]:
    X = rng.randn(m, T, D)
    W, b = init_lstm(D, H, seed=0)
    h0, c0 = np.zeros((m, H)), np.zeros((m, H))
    t_loop = best_time(lstm_forward, X, h0, c0, W, b)
    t_fast = best_time(lstm_forward_fast, X, h0, c0, W, b)
    with threadpool_limits(limits=1, user_api="blas"):
        t_loop1 = best_time(lstm_forward, X, h0, c0, W, b)
        t_fast1 = best_time(lstm_forward_fast, X, h0, c0, W, b)
```

- `def best_time(fn, *args, repeats=7):` — ***args** collects any number of positional arguments into a tuple; **fn(*args)** passes them on.
- `min(times)` — the fastest of several runs is the one least disturbed by other work on the machine.
- `[x for x in threadpool_info() if x["user_api"] == "blas"]` — keep only the entries describing the matrix library.
- `with threadpool_limits(limits=1, user_api="blas"):` — inside this block the matrix library uses one thread; the setting is restored when the block ends.
- `rows.append((f"D={D}, H={H}", ...))` — a tuple of a label and two speed-ups.

Timing one product on its own

CODE

```
x_h = rng.randn(32, 256); W_big = rng.randn(256, 512)
h_only = rng.randn(32, 128); W_h = rng.randn(128, 512)
reps = 300
timings = {}
for label, limit in [("default threads", None), ("one thread", 1)]:
    with threadpool_limits(limits=limit, user_api="blas"):
        for name, A, B in [("[x, h] @ W  (32x256 @ 256x512)", x_h, W_big), ("h @ Wh      (32x128 @
            128x512)", h_only, W_h)]:
            start = time.perf_counter()
            for _ in range(reps):
                A @ B
            timings[(name, label)] = (time.perf_counter() - start) / reps * 1e6
```

- `for label, limit in [("default threads", None), ("one thread", 1)]:` — limits=None leaves the library's default in place.
- `for _ in range(reps): A @ B` — the same product many times; the result is thrown away.
- `/ reps * 1e6` — average seconds per call, converted to microseconds.
- `timings[(name, label)]` — a dictionary keyed by a pair of strings.

Segment 6 — demo_long_sequences

Running 200 steps by forget bias

MATH

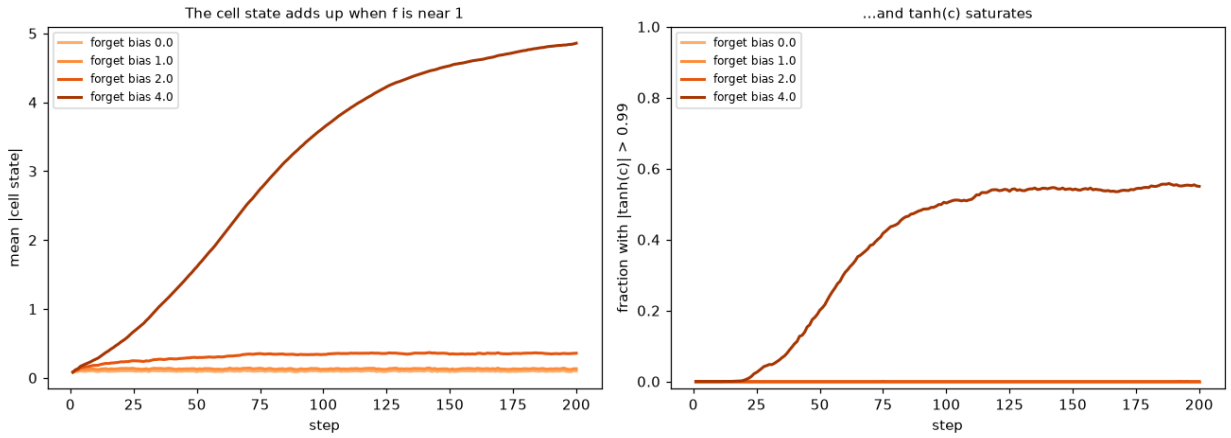
$$saturated \Leftrightarrow |\tanh(c)| > 0.99 \Leftrightarrow slope\ 1 - \tanh(c)^2 < 0.02$$

CODE

```
for fb in [0.0, 1.0, 2.0, 4.0]:
    W, b = init_lstm(1, H, seed=3, forget_bias=fb)
    Hs, Cs, _ = lstm_forward(X, np.zeros((m, H)), np.zeros((m, H)), W, b)
    saturated = np.abs(np.tanh(Cs)) > 0.99
    curves[fb] = (np.abs(Cs).mean(axis=(0, 2)), saturated.mean(axis=(0, 2)))
    for t in [0, 9, 49, 199]:
        print(f"{fb:>7}{t + 1:>6}{np.abs(Cs[:, t]).mean():>10.3f}{np.abs(Cs[:, t]).max():>10.2f}"
              f"{saturated[:, t].mean():>11.1%}{np.abs(Hs[:, t]).mean():>10.3f}")
```

- `np.abs(np.tanh(Cs)) > 0.99` — a True/False array: which units, at which steps, are saturated.
- `np.abs(Cs).mean(axis=(0, 2))` — average over examples (axis 0) and units (axis 2), leaving one number per step.
- `saturated.mean(axis=(0, 2))` — the mean of True/False values is the fraction that are True.
- `{value:>11.1%}` — the % format multiplies by 100 and adds a percent sign.

REAL GRAPH



Real graph from the script: mean |c| and the saturated fraction at every step.

Segment 7 — demo_memory

Measuring the cache

MATH

LSTM: (D + 6H) numbers per example per step RNN: (D + H)

CODE

```
lstm_parts = {k: cache[k].nbytes for k in ["X", "H", "C", "F", "I", "G", "O"]}
rnn_parts = {"X": X.nbytes, "H": Hr.nbytes}
print(f"\n{'array':<7}{'LSTM (MB)':>11}{'RNN (MB)':>10}")
for k in ["X", "H", "C", "F", "I", "G", "O"]:
    print(f"{k:<7}{lstm_parts[k] / 1e6:>11.2f}{(rnn_parts[k] / 1e6 if k in rnn_parts else
        0):>10.2f}")
total_l, total_r = sum(lstm_parts.values()), sum(rnn_parts.values())
```

- `cache[k].nbytes` — the real memory an array occupies, in bytes.
- `{k: ... for k in [...]}` — a dictionary comprehension: array name to its size.
- `rnn_parts[k] / 1e6 if k in rnn_parts else 0` — a one-line if/else inside the f-string: 0 for arrays an RNN does not have.
- `sum(lstm_parts.values())` — add up every value in the dictionary.

Segment 8 — The diagram

draw_unrolled_diagram

CODE

```
xs = [2.6, 6.5, 10.4]
for k, x in enumerate(xs):
    ax.add_patch(FancyBboxPatch((x - 1.2, 1.7), 2.4, 1.9, boxstyle="round,pad=0.05",
        facecolor="#F2F2F2", edgecolor="#333333", linewidth=1.3))
    ax.text(x, 2.95, f"LSTM step t={k}", ha="center", va="center", fontsize=10.5,
        fontweight="bold")
    ax.text(x, 2.45, "lstm_step_forward\n(Day 63)", ha="center", va="center", fontsize=8.5)
    ax.text(x, 0.55, f"x[t={k}]", ha="center", va="center", fontsize=10, color="#08519c",
        fontweight="bold")
    arrow(x, 0.8, x, 1.62, "#08519c")
    arrow(x, 3.68, x, 4.3, "#08519c")
    ax.text(x, 4.5, f"H[:, {k}], C[:, {k}] stored", ha="center", va="center", fontsize=8.5)
ax.text(0.45, 3.3, "c0", ha="center", va="center", fontsize=10, color="#a63603",
    fontweight="bold")
ax.text(0.45, 2.05, "h0", ha="center", va="center", fontsize=10, color="#08519c",
    fontweight="bold")
arrow(0.7, 3.3, 1.35, 3.3, "#a63603", 2.4); arrow(0.7, 2.05, 1.35, 2.05, "#08519c")
for x0, x1 in zip(xs[:-1], xs[1:]):
    arrow(x0 + 1.25, 3.3, x1 - 1.25, 3.3, "#a63603", 2.4)
    arrow(x0 + 1.25, 2.05, x1 - 1.25, 2.05, "#08519c")
    ax.text((x0 + x1) / 2, 3.47, "c", ha="center", fontsize=9, color="#a63603")
    ax.text((x0 + x1) / 2, 2.2, "h", ha="center", fontsize=9, color="#08519c")
```

- `for k, x in enumerate(xs):` — step number and box centre together.
- `FancyBboxPatch((x - 1.2, 1.7), 2.4, 1.9, ...)` — a rounded box 2.4 wide and 1.9 tall, its corner placed so it is centred on x.
- `f"LSTM step t={k}"` — the step number written into the label.
- `zip(xs[:-1], xs[1:])` — neighbouring pairs: (first, second), (second, third), for the arrows between boxes.

Quick Syntax Glossary — new or reinforced constructs

- `(np.zeros(shape) for _ in range(4))` — four separate arrays; a = b = np.zeros(...) would be one array with two names.
- `mask=None ... if mask is None:` — an optional argument with a sensible default filled in.
- `m_t * new + (1 - m_t) * old` — keep or replace, row by row, in one line.
- `dict(X=X, H=Hs, ...)` — build a dictionary from keyword arguments.
- `X.reshape(m * T, D)` — treat every step of every sequence as one row.
- `A.transpose(1, 0, 2).copy()` — reorder axes and make the result contiguous in memory.
- `def f(fn, *args): fn(*args)` — pass a function and its arguments along to be called later.
- `with threadpool_limits(limits=1, user_api="blas"):` — temporarily limit the matrix library to one thread.
- `array.mean(axis=(0, 2))` — average over two axes at once.
- `array.nbytes` — how much memory an array uses.
- `zip(xs[:-1], xs[1:])` — walk through neighbouring pairs of a list.

End of Day 64 syntax reference. For the experiments and what they showed, see the Day 64 complete guide.